# Texas Instruments Car Cup Spring 2024

*Demetrios Giotis & Dara Heydarpour*

Rochester Institute of Technology

**ABSTRACT**

The computer engineering department at Rochester Institute Technology holds an annual Texas Instruments Car competition. The competition allows students the opportunity to display their knowledge acquired from the semesters coursework to develop a program for a car to navigate around an unknown track. The car project was structured into three distinct milestones, culminating in a competitive time trial during the competition. The main components used in the vehicle include a MSP432 Microcontroller from Texas Instruments, two DC motors to drive the car, a servo for steering the front two wheels, and a 128-pixel line scan camera used to detect the track. Proper initialization and functionality of all components must be successful in order to complete the project.

## 1. INTRODUCTION

The Texas Instrument (TI) Car Cup is a competition hosted at Rochester Institute of Technology (RIT) at the end of each semester for the Interface and Digital Electronics (IDE) class. The competition is meant to be the culmination of what the students have learned throughout the semester in the IDE class by programming a model car to drive around a track. This car must be completely autonomous and utilize a line-scan camera to navigate its way around a randomly created track as quickly and efficiently as possible. Each car in the competition had the same hardware to use including the previously mentioned line-scan camera, two DC motors, a servo motor, the programmable MSP432 microcontroller, the car frame and wheels, a battery, and other various items. The car used in the competition would be built with each of these parts and has its complete and constructed state shown in Fig. 1.

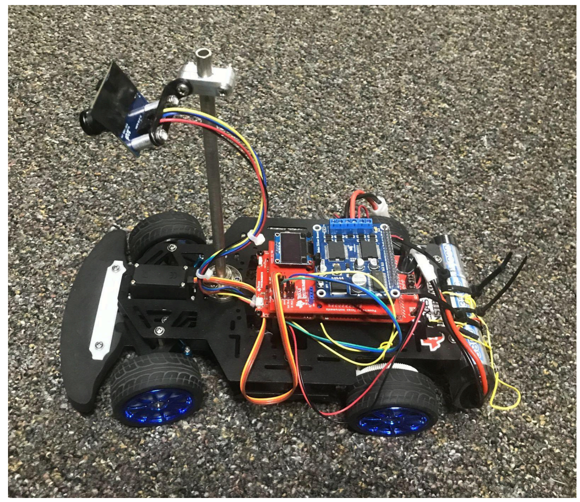

**Fig. 1.** TI Cup Car

Each team involved in the competition had to follow the same set of rules on the racing day. Each team would be given three attempts to have their car complete a single lap around the track, with the completed lap being the final recorded time. Each team would be given five minutes to test their car on the track and be able to make adjustments before handing their cars over, at which point no more alterations could be made to the code on the MSP432 board. Each race for a given team would take place successively with only modifications between attempts being changing the battery or changing the angle of the camera.

While different approaches to controlling the car were possible, the most commonly used algorithm was Proportional-Integral-Derivative (PID) control. PID could be used to control the steering of the car to keep it centered on the track as well as control the car's speed to keep it from moving too quickly around turns allowing it to still move quickly while maintaining control to keep it from going off the track.

The rest of this paper is as follows. After the introduction, Section 2 contains the Bill of Materials (BOM) of the car project, including and introducing each of the components used within. Section 3 includes a detailed look at the algorithms methods utilized in the software and PID control of the car. Section 4 contains the analysis and results of the competition. Finally Section 5 gives the closing remarks on the project as a whole where is followed by the references used throughout the paper.

## 2. BACKGROUND

### 2.1. Milestones

The development process of the autonomous vehicle was organized using a pipelining approach, which consisted of three distinct milestones. The first milestone was to navigate the car around an oval shaped track in both directions. In the second milestone, the car was tested to see whether it could traverse around a figure eight track bidirectionally. Milestone 2 additionally implemented carpet detection to stop the car if it left the track. The final milestone consisted of implementing a proportional integrative derivative control algorithm to drive the steering. In addition, the car must demonstrate the ability to variably control speed.

### 2.2. Bill of Materials

There are many parts that are used to create a complete TI car that is used in the competition. Each component was necessary to have complete and correct functionality for the car. Some of the key components include the line-scan camera to read where the light coming from the track, two DC motors to control the speed of the car, a servo motor to control the turning of the car, the MSP432 microcontroller to allow the components to use software to work in tandem, and the car chassis and wheels to hold the components and move along the track. The BOM for all of these components is shown in Table 1.

TABLE 1. Bill of Materials [1]

| Item | Cost (USD) |
|---|---|
| Parallax TSL-1401 Line Scan Camera | 80 |
| Steering Arm for Servo | 17.99 |
| RB-WAV-77 | 28.99 |
| ROB0170 | 98.75 |
| KIT0167 | 25 |
| HM-10 Bluetooth Module | 10.99 |
| Batteries | 39.99 |
| Battery Charger | 19.99 |
| Camera Ribbon Cable | 6.69 |
| Power Connector | 8.68 |
| Zip Ties | 18.99 |
| Total | 363.05 |

### 2.3. MSP432 Microcontroller

The TI MSP432 microcontroller family consists of high performance, low cost embedded systems that provide a capability for a wide variety of processes. The microcontroller is based on an ARM Cortex-M4F core, which allows for minimum memory requirement, a reduced pin count, and a low power consumption. Fig. 2 displays the high level block diagram of the microcontroller, showing the capabilities available. The functionality of components used in the autonomous vehicle are dependent on the initialization of certain MSP432 modules. Peripherals such as UART, ADC, Timer32, and TimerA are some of the modules used to interface different sensors and actuators during the development process of the autonomous vehicle. The versatility provided by the MSP432 microcontroller family allows it to be an essential component for the project.

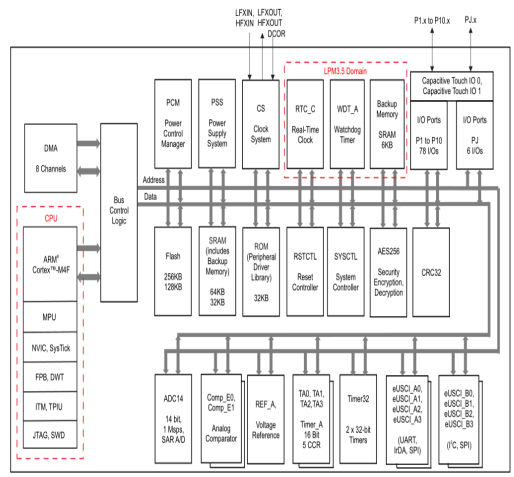


**Fig. 2.** MSP432 High Level Block Diagram [2]

### 2.4. Line Scan Camera

The line-scan camera (Parallax TSL-1401 Line Scan Camera) used for the car is a one-dimensional camera that has 128 photodetectors that each output an integer value. These values need to be converted from an analog to digital voltage using the MSP432s analog to digital converter (ADC) to be converted to a 14-bit resolution, ranging from 0 to $2^{14}$ - 1 or 16383. This single number then could be used to determine the dim areas of the track, values around 3000 or lower, or bright areas, values around 15000. This could be used later to determine where the track is.

### 2.5. DC Motors

The car utilized two DC motors which could drive it forward or backwards. Each motor would spin based on the voltage passed into them, varying in both direction and magnitude. A higher voltage caused the motors to speed up while a lower voltage would cause them to slow down. Because of this Pulse Width Modulation (PWM) was used to control the speed of the motors, and by extension the car, by utilizing different duty cycles. A higher duty cycle would result in the

PWM signals being high for a longer period of time and increase the average voltage and speed the motors spin at. The direction for each of the DC motors was controlled using an H-Bridge circuit with GPIO pins, which is represented in Fig. 3.

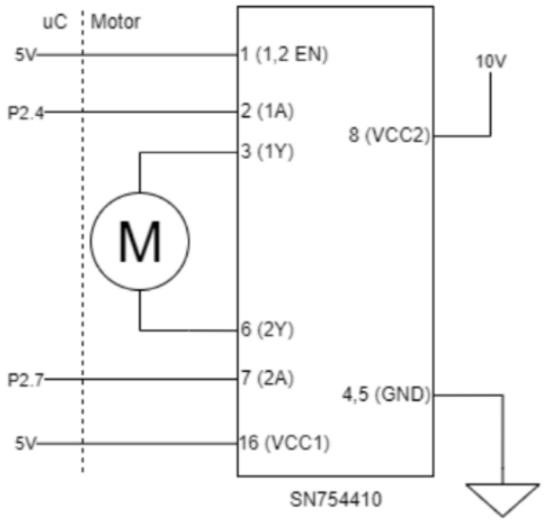


**Fig. 3.** DC Motor H-Bridge Schematic [1]

### 2.6. Servo

The main way to control the direction of the car was done by utilizing a servo motor. The servo motor was connected to the front two wheels of the car and when turned, changed the angles of the wheels. The servo was controlled with a 20ms PWM signal that would change the direction of the servo depending on the duty cycle. A 10% duty cycle was used to turn the servo to the leftmost position, while a 5% duty cycle was used for the rightmost position. A duty cycle of 7.5% would result in the servo being straight and not turning the car. The visualization for this duty cycle is shown in Fig. 4.

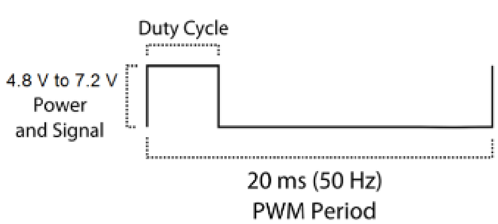


***Fig. 4.*** Servo Duty Cycle [3]

### 2.7. Differential Steering

Differential steering was also utilized early on as a way to turn the car faster. Rather than just rely on the servo to turn the car alone, differential steering changed the speed of each of the motors to turn faster as well. During a turn the motor on the same side as the turn would speed up while the other motor would slow down causing the car to turn quicker than utilizing the servo alone. Depending on the angle of the turn each motor would speed up or slow down by a different amount.

## 3. PROPOSED METHOD

### 3.1. Steer to Center

The first process designed to create the vehicle control algorithm is track detection. The coordinates of the vehicle relative to the track are determined by using the Parallax TSL-1401 line scan camera. The TSL-1401 line scan camera contains an array of closely spaced pixels that are sensitive to light. The pixels are stored as integers and determine bright and dim spots on the track. The camera captures an image by scanning across the track one line at a time. Each time the camera scans, the light is integrated over a period known as the integration time. The default integration time of the scan input is set at 7.5 ms or 133 Hz, however, in the design of the autonomous vehicle, the integration time is increased to 12.5 ms or 80Hz. Increasing the integration time leads to increased sensitivity, allowing for each pixel to collect more light during each scan. Longer integration times also lead to sharper image quality and improved range of light intensity. The trade off of increasing the integration time is that the image acquisition is slower. The output of the line scan camera produces an analog output. The output is digitized by using the ADC14 module of the MSP432 microcontroller. The analog to digital output produces values in the range of 0 to 65,535; corresponding to the magnitude of light from each pixel. In practice this range produces values from 0 to 15,000 where 15,000 signifies the brightest areas, and 0 signifies the darkest. The digital data from the camera is reconstructed to be used to determine track position. The track position is determined by first creating a fake center line. An imaginary grid is created to break the track up into a coordinate grid. The fake center line determines the index of the track that the car must position itself towards. The center of the track is used as the index to access a specific value from the camera line data in order to determine the center of the track. The value accessed from the camera is compared to a list of threshold values. These threshold values determine the action the car must complete given the track center is known. The threshold values were estimated by tracking the line scan camera data when shown different amounts of light over different distances. Since the integration time was determined as 12.5 ms, the threshold values needed to be determined so that the car is not required to change states in under 12.5 ms.

The threshold car state values were determined by placing a piece of paper 5 feet away from the camera, and shifting it across the horizontal axis. The accuracy of these tests were low as the amount of light is different from the actual track. The tests were run in this fashion because the microcontroller is not compatible with Apple computers, and therefore the project had to be developed in the laboratory. Table 2 shows the threshold values and the instruction corresponding to the range.

TABLE 2. Threshold Ranges and Resulting Car Action

| Threshold Values | Car State Ranges | Car Action |
|---|---|---|
| 0 | 0 - 2400 | Off track |
| 2400 | 2400 - 4000 | Track Edge |
| 4000 | 4000 - 7000 | Turning |
| 7000 | 7000-10,000 | Turn Incoming |
| 10,000 | 10,000 - 12,000 | Normal |
| 12,000 | 12,000+ | Straight |

If the car is on the track edge, the program is intended to turn the servo to the maximum amount, ensuring the car stays on track. If the car is off track, it should stop. In any of the other cases, the car will steer to center using PID.

### 3.2. Carpet Detection

To complete the requirements set for milestone 2, the car had to successfully show the car stopping once it detected that it was on the carpet. This was done by comparing the magnitude of line scan camera data at the center camera index to a predetermined constant (2400). If the magnitude at the center visual of mass was less than the predetermined value of 2400, the car was off track. If the car is off track, it disables both DC motors in order to stop the car promptly.

### 3.3. PID Steering Algorithm

To complete milestone three and have an improved steering algorithm, the PID control algorithm was implemented to the design. The algorithm takes the desired difference in angle to the center of the track, and takes the actual angle of the track center. The difference is then calculated to set the steering angle of the servo. Equation 2 shows the algorithm derived to output the optimal number of degrees the servo should turn to reach the center of the track. A negative output from the equation means that the car should turn left, with a positive output intending a right turn. Values for $K_p$, $K_i$, and $K_d$ were determined through testing.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2)$$

Requiring the microcontroller to compute integrals is extremely time consuming. Instead, another equation was used to estimate the track values.

$$S = actual + K_p(e + K_i(e + e_1) IT_1 + K_d(e - IT_2 * e_1 + e_2) \quad (3)$$

The variable S is the number of degrees that the servo needs to turn. $IT_1$ and $IT_2$ are integration time values which are determined through testing where $IT_1$ is 0.5ms and $IT_2$ is negative 2 ms. The error value ($e$) is derived by taking the difference of the current track position, to the intended track position. The other error values ($e_1$, $e_2$) are previous error values. Each time PID steering is called, the array of errors is updated and the newest error is stored at index 0.

### 3.4. Differential Steering Algorithm

A differential steering algorithm was created to make the car steer more smoothly. Differential steering combines the movement of the DC motors, with the Servo steering simultaneously. Differential steering works by slowing the rotating speed of the inner angle of the car depending on the severity of the turn. Through testing, it was determined that the most optimal solution to differential steering was to decrease the speed of the inner wheel by 30% and increase the speed of the outer wheel by 5%. During the second milestone, the algorithm used an increase of 10% and a decrease of 10%, however, this created problems when the car attempted to go through snake tracks.

### 3.5. Car Testing Methods

The car was tested in real time to check the functionality of the car while on the track. The first function was the initialization of LED's to check the state of the car in real time. This allowed for the intended action of the car to be known at all times. The UART peripheral was used to debug the code and see if the program followed the intended path.

### 3.6. Speed Control

For the completion of milestone three, the car was required to demonstrate variable speed control. The car should significantly increase speed on straight aways and decrease speed to the minimum on the track edge. For turning, turn incoming, and normal states, the car will speed up or down based on the number of degrees the servo is turning. The speed is set by changing the duty cycle of the DC motor. The minimum speed was set to 0.25 and the maximum speed was set to 0.35. These values are conservative to the rest of the competition. The design choice to keep the speeds relatively low was made to offset the approximated camera threshold values.

The speed is scaled by dividing the track center value by a predetermined value, and then multiplying the scalar to the current speed. The predetermined value was determined through testing (7000). Decreasing this predetermined value will increase the scaler, and therefore increase the speed of the car.

## 4. RESULTS

Through countless interactions of tests, the design was fully developed and ready for competition. Prior to the race, the car had low expectations when it came to placing high in the competition, however, the functionality of the car seemed flawless. However, on the day of the race, the car failed to finish a lap on the track in all three attempts.

Many challenges were encountered during the project. The most significant challenge was testing the camera without having access to the track. The camera values are read through the values being displayed on the UART. The UART requires the MSP432, which is not compatible with Apple computers. Instead, pieces of paper were placed on the floor to mimic track pieces. Black lines were drawn using Sharpie in order to mimic the track tape.

This approximation works up to a certain point because there is a significant color difference between the track and a sheet of paper. Working exclusively at the Rochester Institute of Technology CMPE-3460 laboratory also limited the number of tests that could be performed. The travel distance between where cars could be tested and the laboratory took approximately five minutes to travel each way. This time significantly adds up over the course of the project and led to significantly less tests than expected.

The car race took place on April 27th, 2024 from 1pm - 2pm at the Ritter Ice Arena. Fig. 5 shows the layout of the track on the day of the race.

***Fig. 5.*** Ritter Track Layout [4]

The Ritter Ice Arena has a large window allowing for sun to come through from the north face of the building. On the day of the race, the sun looming into the building made it so the camera outputted different values than when the car was tested in the IOT lab due to the difference in light. The travel distance between the Ritter arena and the laboratory takes approximately fifteen minutes of travel each way, meaning only two tests were able to be run on the day of the race before submitting the car.

The car ended up failing because the camera threshold values did not work for the track when it was in Ritter Ice Arena. The car could not determine the correct state to be in and would not be able to enter the right mode. On the first attempt, the car made it to the hill before not being able to make it over. The car determined that the hill was an incoming turn and set the speed to be under 35% duty cycle. The motor being at a 35% duty cycle is the minimum speed it needs to be to get over the hill. Since the car state output was determined as turn incoming, the speed was scaled to be under a 35% duty cycle.

During the second run, adjustments were made to make the car go faster. On this attempt, the car made it past the first loop but crashed on a steep turn. This error is attributed to standing in the wrong position, leading to a shadow being displayed on the track. When the camera scanned the shadow, it changed the car state and made the car go off track.

In the third run, no adjustments were made because the car had made it far at a good speed in the previous run before crashing due to human error. In this run, the car failed almost immediately without even getting to the first turn. This can be attributed to a bad camera reading.

If the autonomous vehicle were to be designed again, a few design changes would be implemented. Currently, the camera integration time is increased, leading to better camera data, but with a longer period. In a future iteration of the project, the integration time would decrease and an algorithm to filter the data would be created. This design choice would make the car more easily adaptable to different environments as it is generating new scans more frequently.

The TI autonomous vehicle was still a success even though it did not complete a lap. The vehicle passed all three milestones when it was in the IOT lab and was successfully able to navigate all possible tracks. Unfortunately external factors led the car to not accomplish the design goal, however, the knowledge gained throughout the experience is critical for future IDE development.

## 5. CONCLUSION

In conclusion, the Texas Instruments Autonomous Vehicle Cup competition at Rochester Institute of Technology presented a valuable learning experience for students in the Interface and Digital Electronics (IDE). Despite encountering challenges related to testing and environmental factors, such as the difference in lighting conditions on race day, the project provided important insights to create a design with all the components created in previous laboratory exercises.

The structured approach of the project, including the three milestones focusing on navigation, carpet detection, and PID control, allowed students to apply theoretical knowledge into practical implementations. The utilization of the MSP432 microcontroller alongside components like the line scan camera and DC motors showcased utilization of all the components in one design.

While the car did not achieve its goal of completing a lap during the competition, the project successfully passed all three milestones prior to the race day. This experience underscores the importance of iterative design, real-world testing, and adaptation in engineering projects. Moving forward, improvements such as optimizing camera integration time and refining algorithms will contribute to enhanced performance and resilience in similar autonomous vehicle applications. The TI cup was beneficial in teaching students how to implement theoretical applications into a practical design and developed students problem solving ability which is a critical skill in real world applications.

## REFERENCES

[1] Alec Paul and Ben Hyman. “The Trials and Tribulations of the Texas Instruments Car Cup”. Rochester Institute of Technology, 2022. [Accessed: 29-Apr-2024].

[2] “MSP432P401R, MSP432P401M Mixed-Signal Microcontrollers,” *ti.com*, Jul. 2016. https://www.ti.com/lit/ds/slas826e/slas826e.pdf [accessed Apr. 30, 2024].

[3] MG996R high torque metal gear dual ball bearing servo - electronicos caldas.” [Online]. Available: https://www.electronicoscaldas.com/datasheet/MG996R-Tower-Pro.pdf. [Accessed: 29-Apr-2024].

[4] C. Vo, Photo of TI Cup Track. 2024.